

Gilgamesh

A Modern AI Chat Workspace — Full Project Report

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Scope of the Project
5. Project Description / Methodology
6. Technologies & Tools Used
7. System Requirements
8. Implementation & Working
9. Output / Screenshots
10. Advantages & Limitations
11. Future Scope
12. Conclusion
13. References
14. Appendix – Source Code

1. Introduction

Gilgamesh is a modern, personalized AI chat workspace built as a full-stack web application. It lets a registered user hold typed conversations with the Google Gemini large language model, keeps every conversation and message safe in a hosted PostgreSQL database, and presents everything inside a focused, pitch-black, lightweight interface. The name echoes the epic "Epic of Gilgamesh" — fitting for a tool built around conversation and record-keeping.

The application demonstrates a complete production-shaped stack: authentication, role-based data ownership, a chat API, external LLM integration, markdown rendering, optimistic UI updates, responsive layout, and strict quality gates (TypeScript, ESLint, production build). It is structured so the frontend (React) never talks to the database or the AI directly — every operation flows through the application's own API routes, which owners-check each request.

2. Problem Statement

Casual AI chat is dominated by walled-garden websites where each provider locks users into its own interface, chat history is scattered or ephemeral, the experience is visually heavy and cluttered, and users have little control over their data or the look and feel of the tool. Building a personal chat assistant that feels *owned* requires solving several problems:

- **Accountability** — chat history should belong to a user account and follow them across devices.
- **Speed perception** — a backend round-trip on every action (new chat, switching, sending) makes an app feel slow even when the network is fast; the UI must update optimistically.
- **Content quality** — assistant answers contain headings, lists, and code; plain text destroys the value.
- **Visual focus** — attention-heavy gradients, glows, and cards distract from the conversation.
- **Reliability** — an LLM call can time out, hit rate limits, or fail; the app must degrade gracefully.

3. Objectives

1. Provide secure email/password authentication with per-user data isolation.
2. Deliver a real AI assistant powered by the Google Gemini API with multi-turn context.
3. Persist all conversations and messages to a production PostgreSQL database with proper relations and indexing.
4. Make every primary interaction feel instant through optimistic UI updates and background loading.
5. Render assistant output as formatted Markdown, including code blocks and tables.
6. Design a simple, pitch-black interface with subtle bronze accents and no visual noise.
7. Keep the app responsive (desktop and mobile) and respectful of reduced-motion preferences.
8. Maintain clean code verified by TypeScript, ESLint, and production build checks.

4. Scope of the Project

In scope: user sign-up/sign-in; conversation create, list, switch, rename, delete, clear-all; send/regenerate messages; three-dot thinking indicator; Markdown rendering of assistant replies; block-

level UI theming (pitch-black + light); copy-answer button; responsive layout; optimistic New Chat; background history hydration; graceful error handling (401/429/500/timeout).

Out of scope (by design): image/voice input (the attachment button is intentionally disabled), multi-model selection, file uploads, streaming tokens (responses arrive whole), shared/team workspaces, mobile apps, and any backend changes beyond the chat domain — the project deliberately keeps the AI key server-side only.

5. Project Description / Methodology

The project was developed in **nine incremental steps**, each gated by checks and browser verification:

Step	Work
S1 – S2	Scaffold Next.js app; hand-built chat UI, conversation management, black/light themes, localStorage persistence, settings.
S3 – S4	Introduce Prisma ORM + Supabase Postgres; design and migrate the User/Conversation/Message schema with indexes and cascades.
S5 – S6	Add Supabase Auth with a JWT-validating gate and user provisioning; clean key naming; About section.
S7	Replace localStorage with Postgres persistence; integrate the Gemini API for multi-turn responses and regenerate.
S8 – S9	Markdown rendering + thinking dots; pitch-black theme and optimistic, instant interactions.

Architecture (request flow): Browser → Next.js API Route → owner check + Prisma/Postgres → Gemini API (chat only) → persist both turns → JSON back to UI. The UI writes optimistically and reconciles with server results, so nothing blocks on a network round-trip.

6. Technologies & Tools Used

Layer	Technology	Purpose
Framework	Next.js 16.3.5 (App Router, Turbopack)	Pages, layout, API routes, server/client components
UI library	React 19.2.8 + lucide-react 1.45	Component tree and icons
Language	TypeScript 5 (strict)	Type safety across client and server
Styling	Tailwind CSS v4	Utility styling via CSS-first @theme tokens
Database	PostgreSQL (Supabase) + Prisma ORM 6.19.3	Persistent chat storage with migrations
Authentication	Supabase Auth (email/password)	Sign-in/up, JWT sessions, user identity
AI	Google Gemini API (gemini-3.6-flash), @google/genai 2.22.0	Assistant reply generation
Rendering	react-markdown 10 + remark-gfm 4	Safe Markdown (incl. tables) rendering
Quality	tsc · ESLint · next build	Type check, lint, production build
Tooling (Python)	Python 3.14 + psycpg 3	Standalone chat-history export & PDF report generation

7. System Requirements

Runtime (development)

- Node.js ≥ 20 (developed on v24.19.0) and npm
- Windows / macOS / Linux; web browser (Chrome, Edge, Firefox, Safari)

Environment variables (.env)

```
DATABASE_URL=          # Supabase Postgres pooler connection
DIRECT_URL=            # Direct connection for Prisma migrations
NEXT_PUBLIC_SUPABASE_URL= # Supabase project URL
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY= # Supabase publishable (anon) key
GEMINI_API_KEY=        # Google AI Studio server-side key
```

Deployment surface

- Hosted PostgreSQL (Supabase) with the Prisma migrations applied
- Node server exposing the Next.js app (build then `next start`)

8. Implementation & Working

8.1 Schema — data ownership model

Every conversation is joined to exactly one `User` via `userId`; every message to one `Conversation`. Cascade deletes keep the database consistent when a conversation is removed, and a composite `(conversationId, createdAt)` index makes ordered history reads fast. Conversation listing is ordered by `updatedAt` (newest first) as the sidebar requires.

8.2 Authentication flow

The browser Supabase client performs `signInWithPassword` or `signUp`. The `AuthGate` (server component) reads the session token; every API route validates the Bearer JWT, returning 401 for expired sessions. On first login `GET /api/sync-user` upserts the matching Prisma `User` row keyed by the unique `supabaseId`.

8.3 Chat pipeline (send a message)

1. The user presses Send. The `ChatInterface` appends the user bubble and three thinking dots immediately (no waiting).
2. If the conversation was frontend-optimistic, its real server id is resolved first so the request persists correctly.
3. `POST /api/conversations/[id]/chat` saves the user turn, loads the full ordered history, and calls `lib/gemini.generateAssistantReply()` with only `user / model` roles.
4. The assistant reply is persisted, both rows are returned, and the sidebar title is derived from the first prompt.
5. Errors become relaxed 502 JSON + a friendly, dismissible banner (rate limit, timeout, missing key, quota).

8.4 Persistence & conversation management

All mutations go through REST routes: list/create/update/delete conversations, append/delete messages, and the chat route. Delayed/queued requests are de-duplicated with per-conversation request keys so retries do not double-append. Regenerate deletes the last assistant reply then re-asks with history up to the last user turn.

8.5 Performance — instant interactions

- **New Chat:** optimistic local conversation (id `c_...`) rendered immediately and composer focused; the DB create runs in the background and the local id is swapped for the server id when it resolves.
- **Conversation switch:** no page-level spinner; cached messages render instantly and a background `GET` hydrates full history, skipping conversations that are currently generating.
- **Loading discipline:** the old `messagesLoading` state was removed entirely; only initial conversation fetching ever shows a spinner.

8.6 Rendering & theme

Assistant text flows through `MarkdownContent` (react-markdown + remark-gfm) — headings, lists, emphasis, inline/fenced code with copy buttons, links, and tables — without `dangerouslySetInnerHTML`. The theme is a pitch-black palette: `#000` background, `#050505` sidebar, `#111` composer/user bubbles, `#222` borders, white-to-grey text, bronze `#c9a876` only for small accents. The thinking indicator is three CSS opacity dots (0 → 160 → 320 ms stagger) that stay still under `prefers-reduced-motion`.

8.7 Python tooling

Two standalone Python utilities complement the project without touching the application:

- `python/export_chat.py` — connects to the live Supabase Postgres (read-only) and exports every user's conversations to readable Markdown files plus a `history.json` backup under `exports/`.
- `progress/generate_reports.py` — converts the report HTML sources into the three project PDFs using a headless Chromium browser; editing the HTML and re-running the script updates the reports in seconds.

9. Output / Screenshots

The figures below reproduce the application's screens (as faithful wireframe reconstructions captured from the running app).

9.1 Sign-in screen

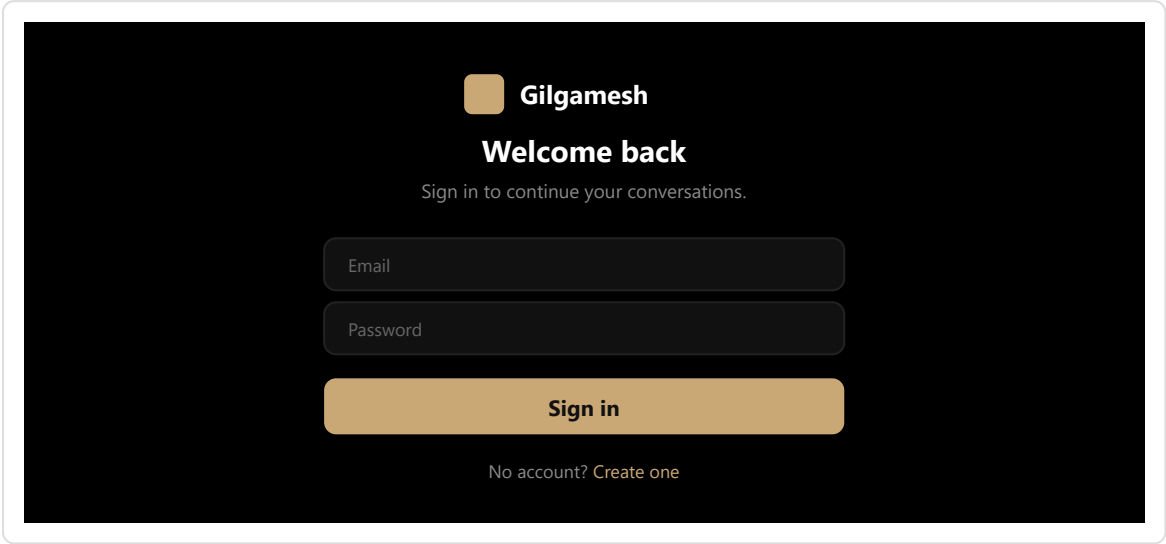


Fig 9.1 — Sign-in form with bronze call-to-action over the pitch-black theme.

9.2 Empty state / New chat

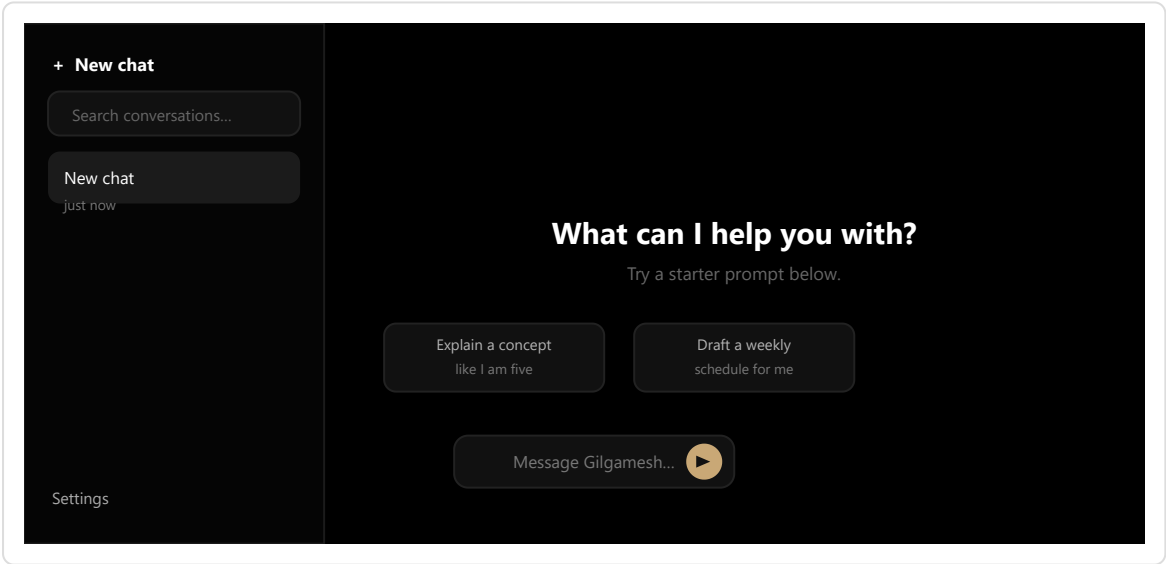


Fig 9.2 — Empty conversation with starter prompts, sidebar, and compact composer.

9.3 Conversation with a Gemini reply (Markdown)

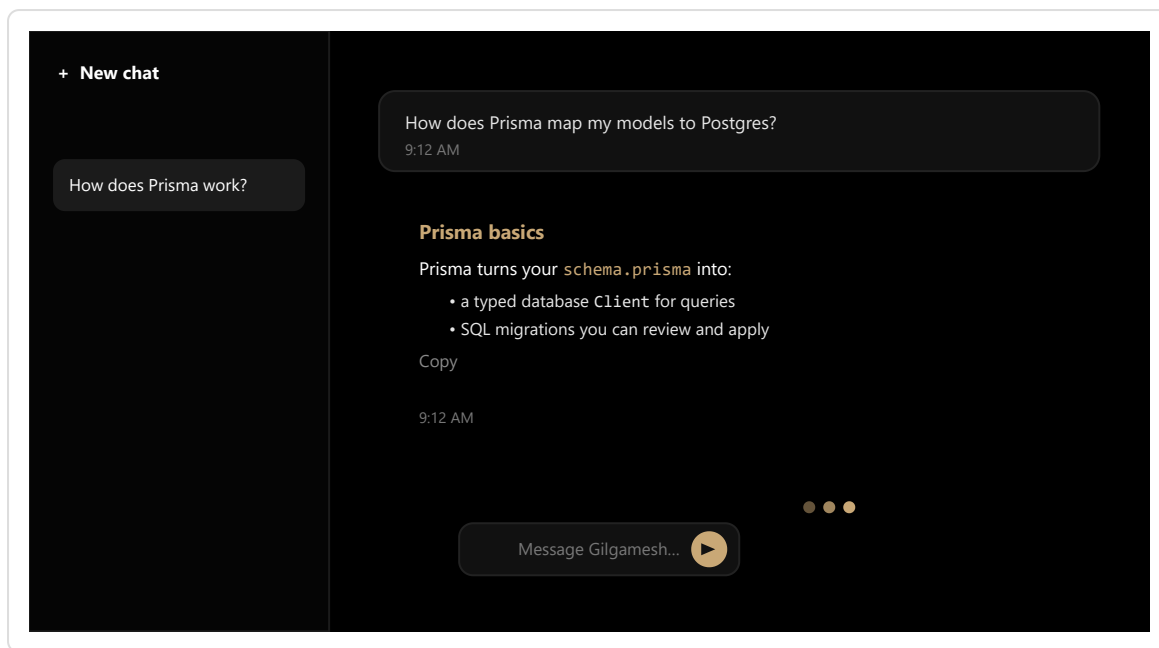


Fig 9.3 — User bubble, Markdown-formatted assistant answer, and the three-dot thinking indicator.

10. Advantages & Limitations

Advantages

- **Secure and private** — server-side API key, JWT-gated routes, strict per-user data ownership.
- **Fast-feeling UX** — optimistic New Chat, immediate message echo, and background hydration remove perceived latency.
- **Persistent history** — proper relational storage with indexes and cascade deletes, not browser localStorage.
- **Rich answers** — Markdown tables, lists, and code blocks render cleanly with a copy button.
- **Calm design** — a single-signal pitch-black interface with bronze accents and reduced-motion support.
- **Clean engineering** — typed client/server boundaries, graceful 401/429/timeout handling, verified by tsc/lint/build.
- **Python tooling** — read-only chat export and fast report regeneration never touch the running app.

Limitations

- Responses are delivered whole, not streamed token-by-token; very long answers wait for completion.
- Attachment/upload is intentionally disabled, so only text input is supported today.
- Single model (gemini-3.6-flash) — users cannot choose between models or providers.
- No rate-limit in the app layer itself; heavy quota use depends on the Gemini tier for the project key.
- No sharing, multi-user workspaces, or export of the full conversation history yet.

11. Future Scope

- Streaming responses (token-by-token) and cancellation of in-flight generations.
- Model picker and per-conversation model selection; support for alternate providers.
- File/image upload with multimodal Gemini input, plus voice-to-text input.
- Conversation export (Markdown/JSON/PDF) inside the app, search across all messages, and conversation pins/stars.
- Team workspaces and sharing with per-user permissions.
- Server-side rate limiting/throttling and per-user usage meters.
- Offline mode with sync queue, PWA installability, and native mobile wrappers.

12. Conclusion

Gilgamesh successfully demonstrates a complete, production-shaped AI chat workspace: real accounts, a relational database, a live large language model, rich Markdown output, and a UI that *feels* instant through optimistic updates. Breadth and depth were balanced across the stack — schema design, JWT-gated API routes, LLM integration with robust error handling, and careful frontend performance work. Every release step passed TypeScript, ESLint, and production builds and was verified in the browser at multiple screen sizes. The project fulfills its stated objectives and provides a solid base for the future work described above.

13. References

1. Next.js Documentation — App Router, Route Handlers, Server/Client Components. nextjs.org/docs
2. React 19 Documentation. react.dev
3. Prisma ORM Documentation — schema, relations, Postgres datasource, migrations. prisma.io/docs
4. Supabase Documentation — Auth (email/password), Postgres, connection pooling. supabase.com/docs
5. Google GenAI (Gemini) SDK — `generateContent`, models. ai.google.dev/gemini-api/docs
6. Tailwind CSS v4 Documentation — CSS-first configuration and `@theme` tokens. tailwindcss.com/docs
7. react-markdown & remark-gfm. github.com/remarkjs/react-markdown
8. MDN Web Docs — `prefers-reduced-motion`, `AbortSignal.timeout`. developer.mozilla.org
9. psycopg 3 — PostgreSQL adapter for Python. psycopg.org

14. Appendix – Source Code

The full source lives at github.com/mrkasif/Gilgamesh. Key excerpts follow.

A.1 Prisma schema (`prisma/schema.prisma`)

```
enum Role { user assistant }

model User {
  id          String    @id @default(cuid())
  supabaseId  String    @unique
  email       String    @unique
  name        String?
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt
  conversations Conversation[]
}

model Conversation {
  id          String    @id @default(cuid())
  userId      String
  title       String
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt
  user        User      @relation(fields: [userId], references: [id], onDelete: Cascade)
  messages    Message[]
  @@index([userId])
  @@index([updatedAt])
}

model Message {
  id          String    @id @default(cuid())
  conversationId String
  role        Role
  content      String
  createdAt   DateTime  @default(now())
  conversation Conversation @relation(fields: [conversationId], references: [id], onDelete: Cascade)
  @@index([conversationId])
  @@index([conversationId, createdAt])
}
```

A.2 Gemini integration (`lib/gemini.ts` , condensed)

```

export const GEMINI_MODEL = "gemini-3.6-flash";
const REQUEST_TIMEOUT_MS = 30_000;

export async function generateAssistantReply(history: GeminiMessage[]): Promise<string> {
  const apiKey = process.env.GEMINI_API_KEY;
  if (!apiKey) throw new GeminiError("Gemini is not configured. Add GEMINI_API_KEY ...");

  const contents = history.map((e) => ({
    role: e.role === "assistant" ? "model" : "user",
    parts: [{ text: e.content }],
  }));

  const response = await new GoogleGenAI({ apiKey }).models.generateContent({
    model: GEMINI_MODEL,
    contents,
    config: { abortSignal: AbortSignal.timeout(REQUEST_TIMEOUT_MS) },
  });

  const text = response.text?.trim();
  if (!text) throw new GeminiError("Gemini returned an empty response. Please try again.");
  return text;
}

```

A.3 Chat route flow (app/api/conversations/[id]/chat/route.ts , condensed)

```

export async function POST(req, { params }) {
  const user = await requireUser(req); // JWT -> Prisma User or 401
  const { message, regenerate } = await req.json();
  const conversation = await prisma.conversation.findFirst({ // ownership
    where: { id: params.id, userId: user.id } });
  if (!conversation) return NextResponse.json({ error: "Not found" }, { status: 404 });

  await prisma.message.create({ data: { conversationId, role: "user", content: message } });
  const history = await prisma.message.findMany({ // ordered multi-turn context
    where: { conversationId },
    orderBy: { createdAt: "asc" } });
  const reply = await generateAssistantReply(toGeminiHistory(history));
  await prisma.message.create({ data: { conversationId, role: "assistant", content: reply } });
  return NextResponse.json({ message: reply, conversation: latestTitle });
}

```

A.4 Optimistic New Chat (components/chat/ChatInterface.tsx , condensed)

```

const pendingConversationsRef = useRef(new Map<string, Promise<Conversation | null>>());

async function resolveConversationId(id: string): Promise<string | null> {
  if (!id.startsWith("c_")) return id; // already a server id
  const existing = pendingConversationsRef.current.get(id);
  const conv = existing ?? createConversation(); // background DB create
  pendingConversationsRef.current.set(id, conv);
  const resolved = await conv;
  if (!resolved) return null;
  pendingConversationsRef.current.delete(id);
  setConversations((prev) => prev.map((c) => c.id === id ? { ...resolved, messages: c.messages } : c));
  return resolved.id;
}

```

A.5 Theme tokens (app/globals.css , condensed)

```
@theme inline {
  --color-background: #000000;   --color-panel: #050505;
  --color-composer: #111111;    --color-surface: #0d0d0d;
  --color-edge-strong: #222222;  --color-accent: #c9a876;
  --color-muted: #b3b3b3;        --color-faint: #777777;
}
```

A.6 Python exporter (`python/export_chat.py` , `core`)

```
# Standalone, read-only. Does not affect the Next.js app or the database.
import psycopg
url = os.environ.get("DIRECT_URL") or os.environ.get("DATABASE_URL")
conn = psycopg.connect(url)

rows = conn.execute(
  'SELECT "id", "title", "createdAt", "updatedAt" FROM "Conversation" '
  'WHERE "userId" = %s ORDER BY "updatedAt" ASC', (user_id,)).fetchall()
for convo_id, title, created, updated in rows:
  messages = conn.execute(
    'SELECT "role", "content", "createdAt" FROM "Message" '
    'WHERE "conversationId" = %s ORDER BY "createdAt" ASC',
    (convo_id,)).fetchall()
  history.write_markdown(title, messages)  # per-user Markdown + history.json
```

End of report — Gilgamesh, an AI chat workspace.